Reflection report

Original backlog

This is how I originally planned the project after the project's specifications. The epic should have been called digtalClock function.

Projects / Reception Management Dashboa

Backlog

Search

JR

Epic

Issues without epic

Structuring the project

digitalFunction

Staff Management Features

Delivery management features

Finalizing project

+ Create epic

Sprint 1: Setup 17 Nov – 24 Nov (4 issues) 0 0 0 Start sprint

RMDFW2-4	Setting up structure of project with files and folders.	STRUCTURIN...	TO DO
RMDFW2-6	Setting up HTML and CSS	STRUCTURIN...	TO DO
RMDFW2-10	Making employee class with children	STRUCTURIN...	TO DO
RMDFW2-20	digitalClock function	DIGITALFUNC...	TO DO

+ Create issue

4 issues | Estimate: 0

Sprint 2: Staff management 24 Nov – 1 Dec (4 issues) 0 0 0 Start sprint

RMDFW2-13	staffUserGet function	STAFF MANA...	TO DO
RMDFW2-14	staffOut function	STAFF MANA...	TO DO
RMDFW2-15	staffIn function	STAFF MANA...	TO DO
RMDFW2-16	staffMemberIsLate function	STAFF MANA...	TO DO

+ Create issue

4 issues | Estimate: 0

Sprint 3: Delivery management 1 Dec – 8 Dec (3 issues) 0 0 0 Start sprint

RMDFW2-17	addDelivery function	DELIVERY MANAGEME...	TO DO
RMDFW2-18	validateDelivery function	DELIVERY MANAGEME...	TO DO
RMDFW2-19	deliveryDriverIsLate function	DELIVERY MANAGEME...	TO DO

+ Create issue

3 issues | Estimate: 0

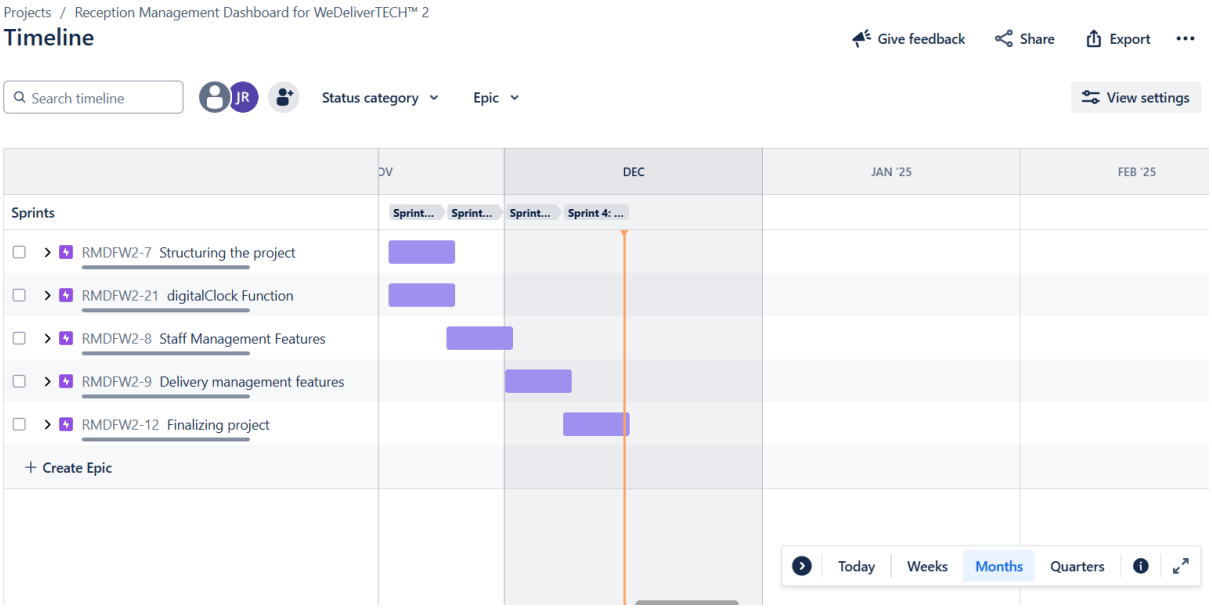
Sprint 4: Finalizing project 8 Dec – 15 Dec (3 issues) 0 0 0 Start sprint

RMDFW2-22	Writing jira report.	FINALIZING PROJECT	TO DO
RMDFW2-23	Checking over project.	FINALIZING PROJECT	TO DO
RMDFW2-24	Ultimate deployment	FINALIZING PROJECT	TO DO

+ Create issue

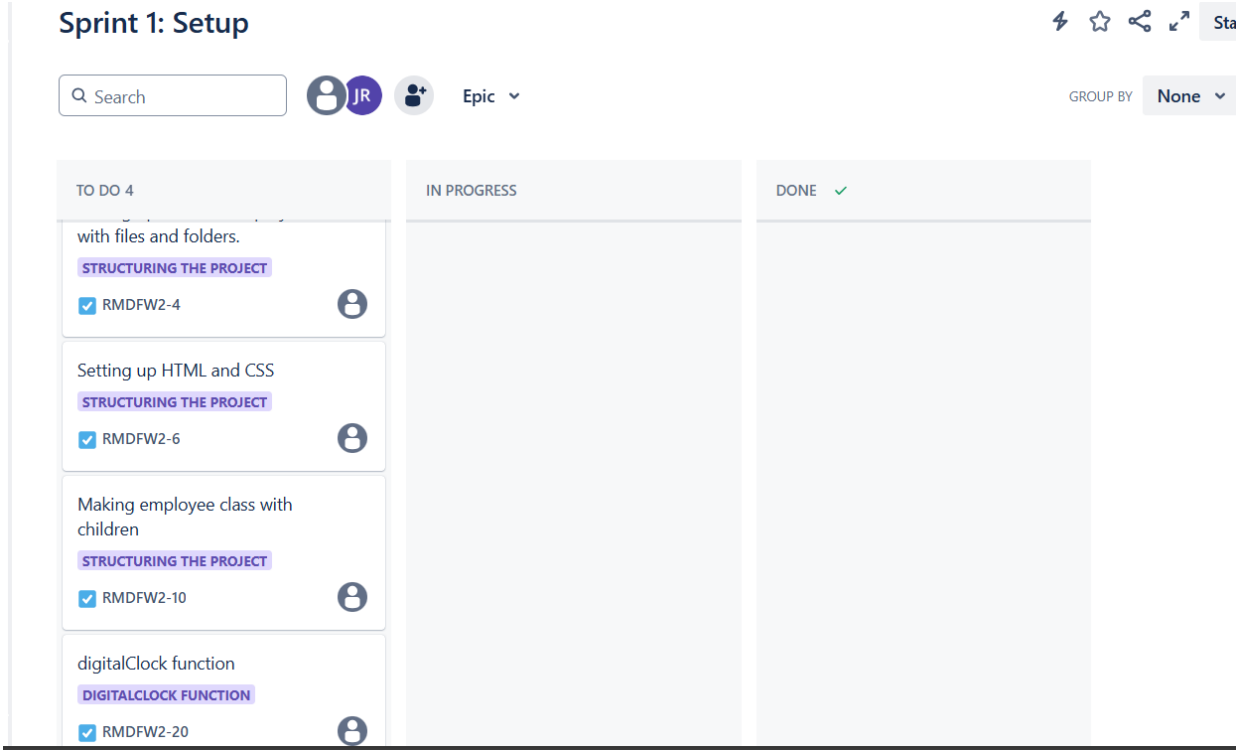
3 issues | Estimate: 0

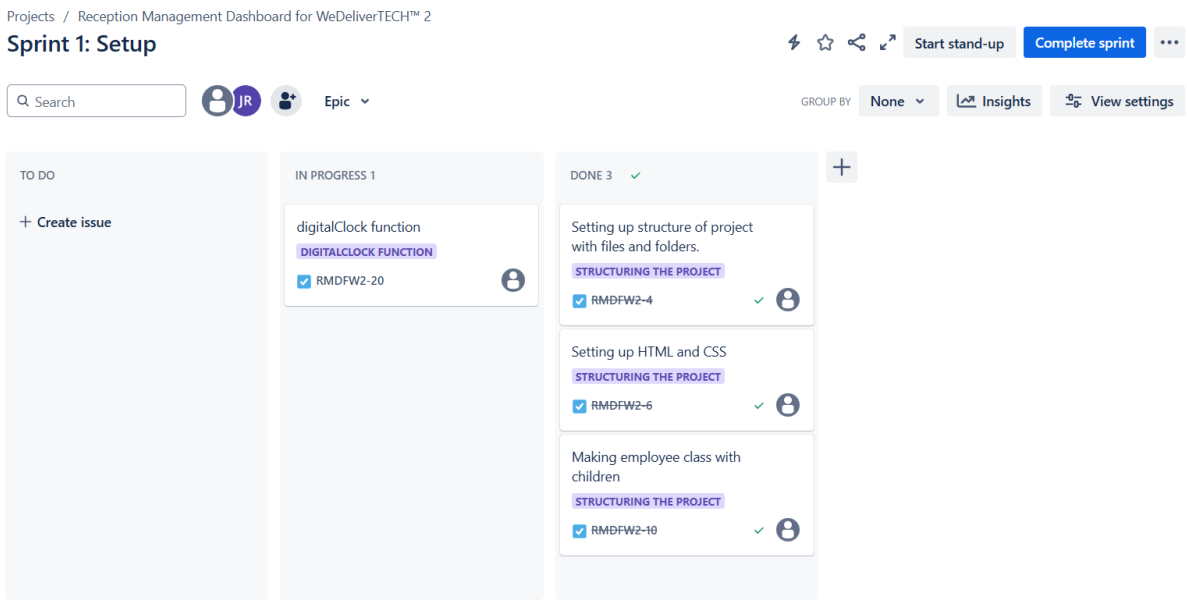
Original timeline



Board

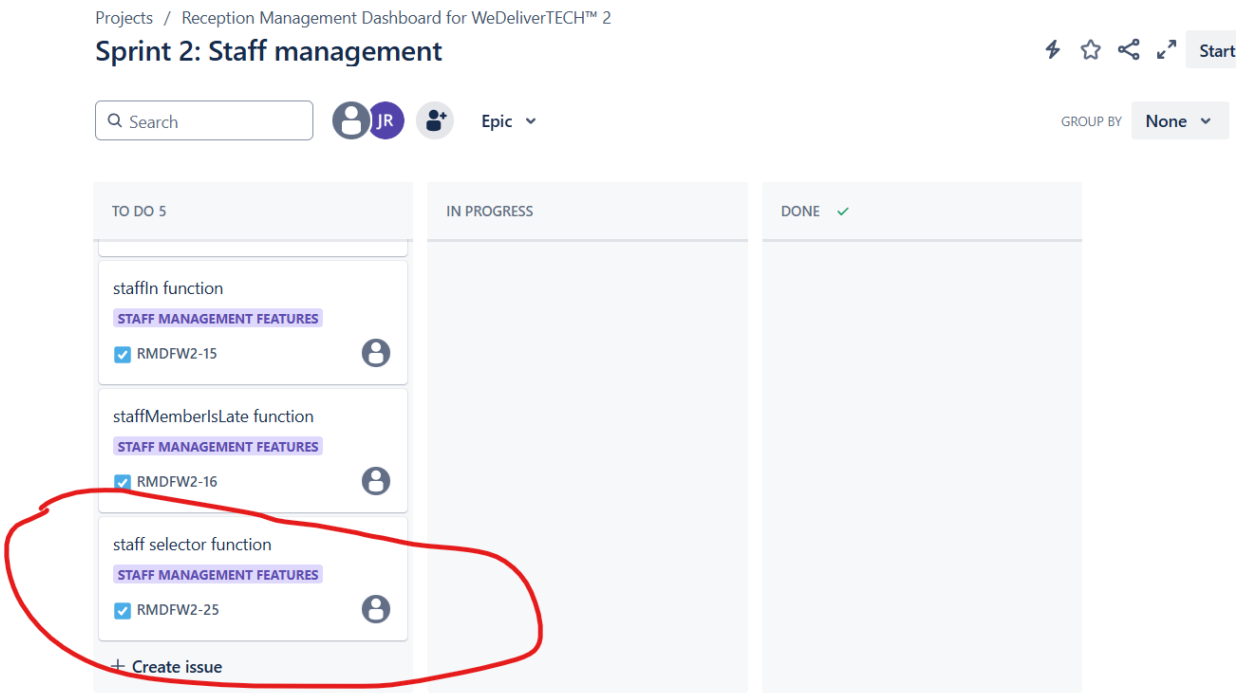
Sprint 1



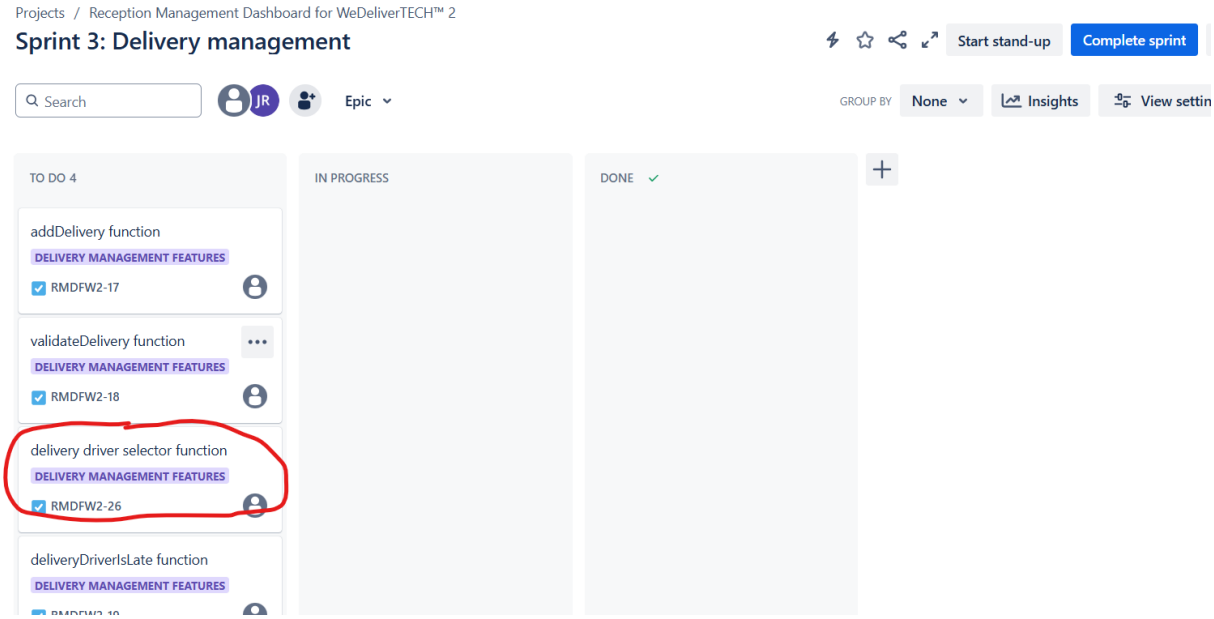


Sprint 2

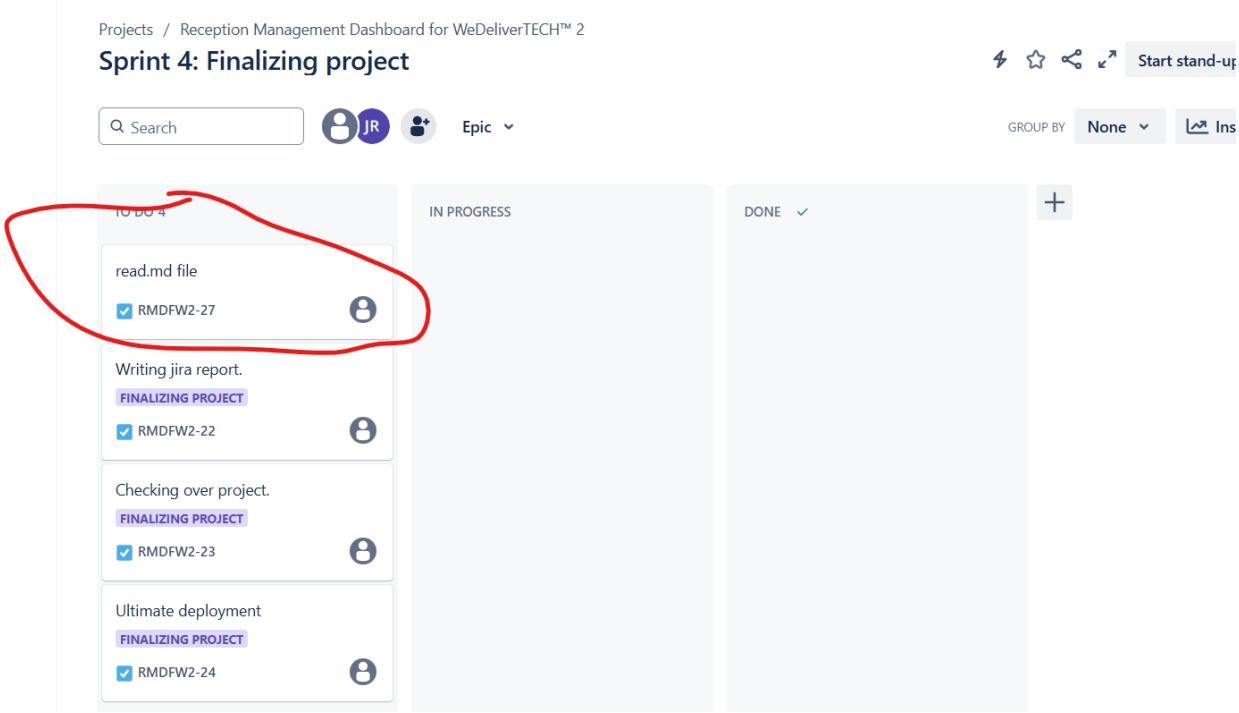
I had to add another function.



Sprint 3



Sprint 4



Summary of project management

I divided the project into four sprints: Setup and Foundation, Staff Management, Delivery Management, and Finalization. Each sprint had clearly defined goals, which helped me stay organized and on schedule. For instance, the second sprint focused on implementing the staffOut and staffIn functions, while the third sprint was dedicated to delivery

management features such as input validation and toast notifications. Allocating the final sprint for review and polishing ensured I could refine and perfect the application.

One of the biggest challenges was ensuring that the staffMemberIsLate and deliveryDriverIsLate functions were called correctly and displayed accurate notifications. Handling time formats posed difficulties, as JavaScript's Date object occasionally introduced discrepancies in expected return times. By using time debugging techniques, I logged timestamps at different points in the program, allowing me to isolate and resolve the issue effectively.

Another challenge was validating user input, such as ensuring that return times followed the hh:mm format and phone numbers contained only digits. To address this, I implemented custom validation checks and added alerts to notify the receptionist of errors. These checks significantly improved the reliability of the system and enhanced the user experience.

Throughout the project, Jira played an essential role in tracking progress. I frequently updated the backlog as new issues arose, ensuring that all tasks were organized and prioritized. For example, when I realized additional functionality was needed, I added these tasks to the relevant epics and sprints without disrupting the overall timeline. This iterative approach allowed me to adapt to changes while maintaining a steady pace.

Overall, this project improved my technical skills, particularly in JavaScript and OOP. Using classes like StaffMember and DeliveryDriver made the code more modular and easier to maintain. Additionally, applying Bootstrap and Font Awesome helped me create a polished, professional UI that adheres to the company’s branding. Incorporating animations and icons further enhanced the user experience, making the application visually appealing and functional.

One significant aspect of the final sprint was focusing on cleaning up the code to ensure it was as efficient and maintainable as possible. This included refactoring repetitive code, removing unused functions, and simplifying logic where possible. For example, I restructured how time calculations were handled in the toast notifications, reducing redundancy and improving performance. This step not only made the codebase cleaner but also prepared it for potential future expansions.

This project also reinforced the importance of testing. By conducting regular tests after each sprint, I identified and resolved issues early, reducing potential delays. For instance, frequent testing of the toast notification system ensured it worked seamlessly for both staff and delivery drivers. Final testing during the last sprint helped me catch edge cases and ensure that the application functioned as intended.

In conclusion, the division of tasks into sprints, combined with Agile principles, made this project manageable and successful. While challenges such as time debugging and input validation were difficult, they provided valuable learning experiences. Cleaning up the code during the final sprint was a crucial step in delivering a well-structured, efficient, and professional application. This project has significantly enhanced my skills in project management, technical development, and problem-solving.

```
staffMemberIsLate() {
  // Ensure expectedReturnTime is valid
  if (!this.expectedReturnTime) {
    return;
  }

  const currentTime = new Date();
  const expectedReturnTime = new Date(this.expectedReturnTime);
  const timeDifference = (currentTime - expectedReturnTime) / (1000 * 60); // Time difference in

  console.log(`${this.name} ${this.surname}: Current Time: ${currentTime}`);
  console.log(
    `${this.name} ${this.surname}: Expected Return Time: ${expectedReturnTime}`
  );
  console.log(
    `${this.name} ${this.surname}: Time Difference: ${timeDifference} minutes`
  );
```